Plataformas de Desenvolvimento

24/25

Mestrado em Informática e Sistemas

1

Ansible

- Motivation
- Ansible architecture
- Concepts
- Modules
- Utilization

2

Motivation

- A lot of tasks are repetitive
  - User creation
  - Initial Setup of a VM
  - Package installation
  - (…)
- Human prone error
  - Hard to detect errors
  - Dificult to repeat
- Concept of Infrastructure as code

3

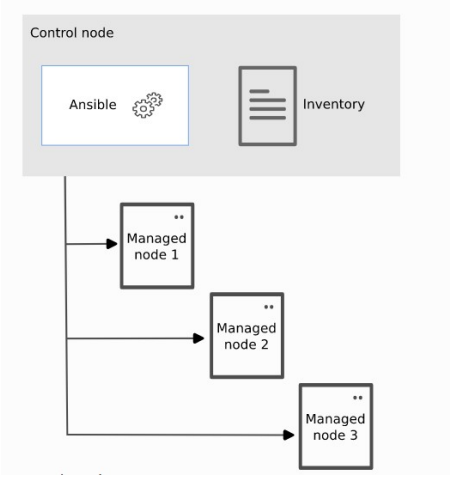
Architecture

Ansible automates the management of remote systems and controls their desired state. A basic Ansible environment has three main components:

Control node
A system on which Ansible is installed. You run Ansible commands such as ansible or ansible-PLAYBOOK on a control node.

Managed node
A remote system, or host, that Ansible controls.

Inventory
A list of managed nodes that are logically organized. You create an inventory on the control node to describe host deployments to Ansible.



4

Architecture

- Communication is made through ssh between control node and serves
- Authentication can be made through user/pwd or ssh keys
- Don’t need agents on the remote server (e.g. Chef, Puppet)

5

Ansible Playbook

- Execute tasks through a pre-defined order
- Execute tasks based on the result of previous tasks
- Execute tasks in parallel
- Can execute the same task on N servers, but ensure sequentiality inside the playbook

6

Advantages

- Automate tasks
- Environment standardization
  - Same setup for distinct environments using the same playbook
- Control
  - The administrator have the decision to when execute the playbook
- Idempotency
  - E.g. If some Webservice should be “Started” in a specific task, Ansible will not start again the WebService

7

Concepts - Inventory

- File with all the information related with a specific environment:
  - IPs
  - Variables
  - Groups of applications
    - E.g. – FrontEnds only should be installed in machine 2 and 3 for a specific environment

8

Concepts - Inventory

all: This is the top-level group that contains all hosts.

- **hosts**: Lists individual hosts and their properties.
  - **webserver1**: The name of the host.
    - **ansible_host**: The IP address of the host.
    - **ansible_user**: The username used to connect to the host.
  - **webserver2**: Another host with similar properties.

- Webservers - Group of webservers
- Databases - Group of databases

```
all:
  hosts:
    webserver1:
      ansible_host: 192.168.1.101
    webserver2:
      ansible_host: 192.168.1.102
    dbserver:
      ansible_host: 192.168.1.103

webservers:
  hosts:
    webserver1:
    webserver2:

databases:
  hosts:
    dbserver:
```

9

Concepts - Playbook

- Yaml format
- Identification of the groups/serves where the tasks will be processed
- Task list
- It is possible to include other playbooks inside our playbook:
  - E.g. - include: add-user-app_admin.yml

```
---
- name: Example Playbook
  hosts: webservers  # Targeting the webservers group
  become: yes  # Run tasks with sudo

  tasks:
    - name: Ensure Apache is installed
      apt:
        name: apache2
        state: present

    - name: Ensure Apache is running and enabled
      service:
        name: apache2
        state: started
        enabled: yes
```

10

Concepts - Playbook

- You can run the same playbook for distinct environments
  - i.e. diferent inventory

11

Concepts - Variables

- Possible to define variables at distinct levels:
  - Host of the Inventory
  - Group of the Inventory
  - Playbook
  - …

```
# Host
[webservers]
webserver1 dbname=web
webserver2 dbname=om

# Group
[webservers:vars]
dbname=webdb
```

```
# Playbook
- hosts: webservers
  vars:
     http_port: 80
```

12

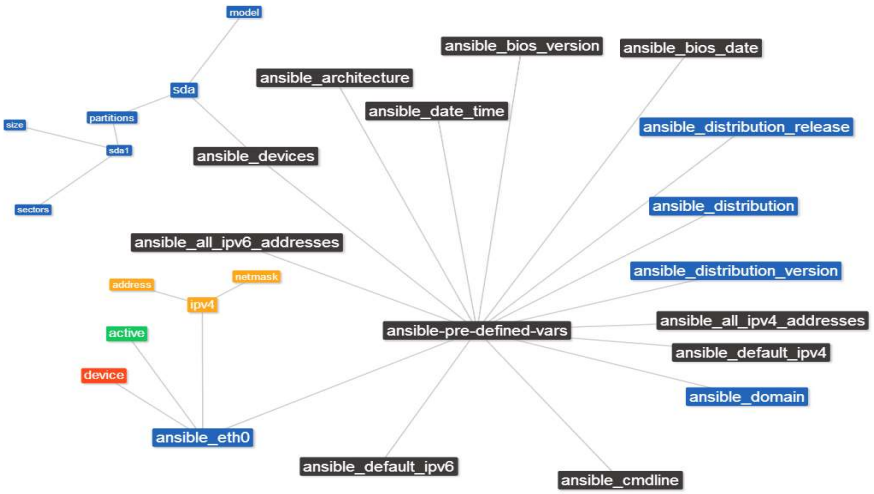
Concepts - Variables

Ansible does apply variable precedence, and you might have a use for it. Here is the order of precedence from least to greatest (the last listed variables override all other variables):

1. command line values (for example, `-u my_user`, these are not variables)
2. role defaults (defined in role/defaults/main.yml) [1]
3. inventory file or script group vars [2]
4. inventory group_vars/all [3]
5. playbook group_vars/all [3]
6. inventory group_vars/* [3]
7. playbook group_vars/* [3]
8. inventory file or script host vars [2]
9. inventory host_vars/* [3]
10. playbook host_vars/* [3]
11. host facts / cached set_facts [4]
12. play vars
13. play vars_prompt
14. play vars_files
15. role vars (defined in role/vars/main.yml)
16. block vars (only for tasks in block)
17. task vars (only for the task)
18. include_vars
19. set_facts / registered vars
20. role (and include_role) params
21. include params
22. extra vars (for example, `-e "user=my_user"`)(always win precedence)

13

Special Variables Available



https://docs.ansible.com/ansible/latest/reference_appendices/special_variables.html

14

Ansible modules

- Ansible have hundreds of modules
  - Each of them have a specific function
  - Possible to create your own modules
  - Large Community
  - …

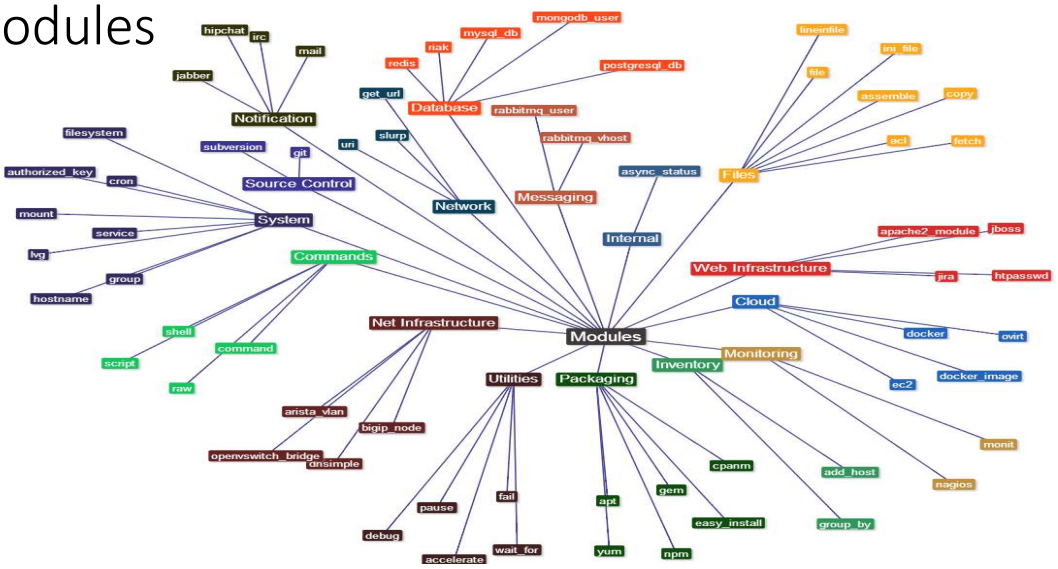
```
name: Get Host File
fetch: src=/tmp/abc.txt dest=/tmp/backup/{{ ansible_host_name }}/
```

```
name: Copy file to remote host
copy: src=file_to_sent dest=/etc/sudoers.d/app_admin mode=0644
```

```
name: Install apache latest version
yum: name=httpd state=latest
```

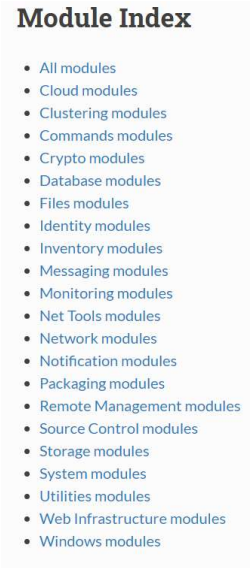
15

Modules



16

Modules



https://docs.ansible.com/ansible/2.9/modules/modules_by_category.html

17

Run ansible playbook

- Command to run a sequence of tasks (or roles)
- Main Options
  - - i : choose the inventory file to process the playbook
  - -v, -vv, -vvv: verbose
- Other options available:
  - -f: number of parallel tasks
  - - u: user to access remote hosts
  - …

18

Run ansible playbook

- Example of running a playbook named uptime.yml for an inventory hosts.dev:

```
$ ansible-playbook uptime.yml –i inventory/hosts.dev -k -v
SSH password:

PLAY [myDesktops] ************************************************************

GATHERING FACTS **************************************************************
ok: [127.0.0.1]

TASK: [Get Uptime] ***********************************************************
changed: [127.0.0.1] => {"changed": true, "cmd": ["uptime"], "delta":
"0:00:00.026360", "end": "2020-02-07 11:43:45.377039", "rc": 0, "start": "2024-
02-24 11:43:45.350679", "stderr": "", "stdout": " 11:43:45 up 23:59,  3 users,
load average: 0,38, 0,25, 0,19"}

PLAY RECAP *******************************************************************
127.0.0.1                  : ok=2    changed=1    unreachable=0    failed=0
```

19

Run ansible playbook

- Example of running the **same** playbook named uptime.yml for an inventory hosts.prd:

```
$ ansible-playbook uptime.yml -k –v –i inventory/hosts.prd
SSH password:

PLAY [myDesktops] ************************************************************

GATHERING FACTS **************************************************************
ok: [www.xpto.pt]

TASK: [Get Uptime] ***********************************************************
changed: [www.xpto.pt] => {"changed": true, "cmd": ["uptime"], "delta":
"0:00:00.026360", "end": "2020-02-07 11:55:41.39999", "rc": 0, "start": "2024-02-
07 11:55:41.66666", "stderr": "", "stdout": " 11:55:41 up 11 days, 28 min, 3
users,  load average: 0,01, 0,02, 0,00"}

PLAY RECAP *******************************************************************
www.xpto.pt              : ok=2    changed=1    unreachable=0    failed=0
```

20

Some exemples – create inventory (1)

Create a host in yaml:

```
root@guest-VirtualBox:/home/guest# cat hosts/dev/hosts.yml
---

all:
  hosts:
    frontend_machine_1:
      ansible_ssh_host:  "localhost"
      ansible_ssh_user:  "guest"
      ansible_ssh_pass:  "guest"
```

21

Some examples – creating inventory(2)

Create a group in a yaml file:

```
root@guest-VirtualBox:/home/guest# cat hosts/dev/groups_yml
##############################################################################
#
#  INVENTORY GROUPS
#
##############################################################################

[frontend_machine]
frontend_machine_1
```

22

Some examples- creating playbooks

Create a playbook that prints all machine characteristics from a specific environment

- in this case only the frontend_machine group

```
root@guest-VirtualBox:/home/guest# cat check_environment.yml
---

- name: Get environment variables
  hosts: frontend_machine
  tasks:
     - name: Print all available facts
       ansible.builtin.debug:
         var: ansible_facts
```

23

Jenkins playbook

```
# Create a jenkins job using basic authentication
- jenkins_job:
    config: "{{ lookup('file', 'templates/test.xml') }}"
    name: test
    password: admin
    url: http://localhost:8080
    user: admin

# Create a jenkins job using the token
- jenkins_job:
    config: "{{ lookup('template', 'templates/test.xml.j2') }}"
    name: test
    token: asdfasfasfasdfasdfadfasfasdfasdfc
    url: http://localhost:8080
    user: admin

# Delete a jenkins job using basic authentication
- jenkins_job:
    name: test
    password: admin
    state: absent
    url: http://localhost:8080
    user: admin
```

```
# Delete a jenkins job using the token
- jenkins_job:
    name: test
    token: asdfasfasfasdfasdfadfasfasdfasdfc
    state: absent
    url: http://localhost:8080
    user: admin

# Disable a jenkins job using basic authentication
- jenkins_job:
    name: test
    password: admin
    enabled: False
    url: http://localhost:8080
    user: admin
```

https://docs.ansible.com/ansible/2.9/modules/jenkins_job_module.html

24

Docker playbook

```
- name: Create Mysql container
  docker_container:
    name: mysql
    image: mysql
    state: started
    recreate: yes
    env:
      MYSQL_ROOT_PASSWORD: "{{ mysql_root_pass }}"
    exposed_ports:
      - "{{ mysql_port }}"
    volumes:
      - db_data:/var/lib/mysql/data
```

https://docs.ansible.com/ansible/2.9/modules/jenkins_job_module.html

25

Running Tasks in DB - playbook

```
- name: Connect to acme database and remove test user privileges from there
  postgresql_user:
    db: acme
    name: test
    priv: "ALL/products:ALL"
    state: absent
    fail_on_user: no

- name: Connect to test database, remove test user from cluster
  postgresql_user:
    db: test
    name: test
    priv: ALL
    state: absent

- name: Connect to acme database and set user's password with no expire date
  postgresql_user:
    db: acme
    name: django
    password: mysupersecretword
    priv: "CONNECT/products:ALL"
    expires: infinity
```

https://docs.ansible.com/ansible/2.8/modules/postgresql_user_module.html

26

Docker playbook – install on Ubuntu

```yaml
- hosts: all
  become: true
  vars_files:
    - vars/default.yml

  tasks:
    - name: Install aptitude using apt
      apt: name=aptitude state=latest update_cache=yes force_apt_get=yes

    - name: Install required system packages
      apt: name={{ item }} state=latest update_cache=yes
      loop: [ 'apt-transport-https', 'ca-certificates', 'curl', 'software-properties-common', 'python3-pip', 'virtualenv', 'python3-setuptools']

    - name: Add Docker GPG apt Key
      apt_key:
        url: https://download.docker.com/linux/ubuntu/gpg
        state: present

    - name: Add Docker Repository
      apt_repository:
        repo: deb https://download.docker.com/linux/ubuntu bionic stable
        state: present

    - name: Update apt and install docker-ce
      apt: update_cache=yes name=docker-ce state=latest

    - name: Install Docker Module for Python
      pip:
        name: docker
```

27

Some examples – running playbooks

```
#Run a playbook with the environment /hosts/dev/
ansible-playbook check_environment.yml -i hosts/dev/
```

```
TASK [Check uptime machine] ****************************************************************************************
changed: [frontend_machine_1]

TASK [debug] *******************************************************************************************************
ok: [frontend_machine_1] => {
    "result": {
        "changed": true,
        "cmd": "uptime",
        "delta": "0:00:00.004136",
        "end": "2021-04-24 19:41:48.379425",
        "failed": false,
        "rc": 0,
        "start": "2021-04-24 19:41:48.375289",
        "stderr": "",
        "stderr_lines": [],
        "stdout": " 19:41:48 up  1:48,  2 users,  load average: 0,45, 0,28, 0,16",
        "stdout_lines": [
            " 19:41:48 up  1:48,  2 users,  load average: 0,45, 0,28, 0,16"
        ]
    }
}

PLAY RECAP *********************************************************************************************************
frontend_machine_1         : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

28

Some examples – running playbooks

```
#Run a playbook with the environment /hosts/dev/
ansible-playbook check_environment.yml -i hosts/dev/
```

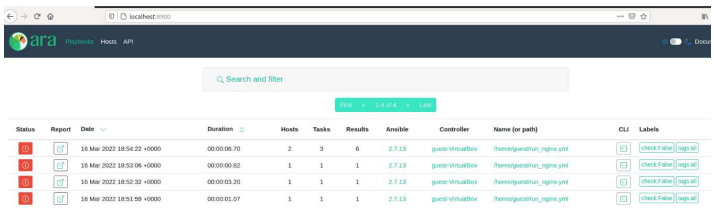
```
PLAY RECAP *********************************************************************************************************
frontend_machine_1         : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

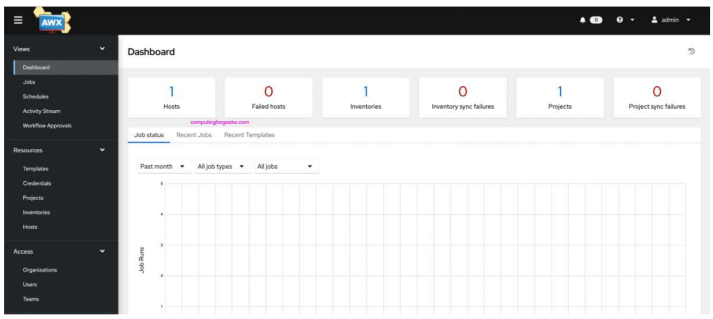
frontend_machine_2         : ok=0    changed=0    unreachable=1    failed=0    skipped=0    rescued=0    ignored=0
```

29

Ansible UI ?

https://github.com/ansible-community/ara





https://github.com/ansible/awx

30

Bibliography

https://www.ansible.com/

https://docs.ansible.com/ansible/2.9/modules/jenkins_job_module.html

https://www.axelerant.com/resources/team-blog/managing-docker-containers-using-ansible

https://github.com/do-community/ansible-playbooks/tree/master/docker_ubuntu1804

https://github.com/RicardoFilipe/ansible-pd